

Parle Tilak Vidyalaya's Association
MULUND COLLEGE OF COMMERCE (AUTONOMOUS)
 DEPARTMENT OF INFORMATION TECHNOLOGY
 T.Y. B.Sc. Information Technology

Name : Sawant Aarush Hemant
 Roll No : 269729
 Course : ASP.NET Core Practical

INDEX

PR. NO	PRACTICAL TITLE	DATE	SIGNATURE (Attendance)	SIGNATURE (Submission)
1.	Creating simple applications and testing applications.	23/6/26	<u>Sawant A</u> 23-06-26	<u>Sawant A</u> 3-7-26
2.	Creating Testing projects.	3/7/26	<u>Sawant A</u> 3-7-26	<u>Sawant A</u> 10-7-26
3.	Creating Applications with data (CRUD).	10/7/26 20/7/26	<u>Sawant A</u> 10-7-26	<u>Sawant A</u> 7-8-26
4.	Creating application using cart.	20/7/26	<u>Sawant A</u> 26-7-26	<u>Sawant A</u> 7-8-26
5.	Implementing Security to application.	07/08/26	<u>Sawant A</u> 7-8-26	<u>Sawant A</u> 10-8-26
6.	Working with URL routing and dependency injections.	10/08/26	<u>Sawant A</u> 10-8-26	
7.	Creating Application with caching.			
8.	Creating RESTful services.			
9.	Creating applications with controllers and views.			
10.	Creating applications using Razor pages and Helper tags.			

Final Submission: _____

Sr.No	Date	Topic	Sign
1	23/6/26	Creating simple & testing applications.	Sian Sandya

Problem Statement: To create Simple and Testing Applications

Q.1

What is ASP.NET Core?

→

It's an open-source cross-platform, high-performance web development framework created by Microsoft. It serves as a specialized extension of broader .NET ecosystem.

Q.2

What is MVE?

→

It stands for Model-View-Controller. It's a software design pattern that separates an application into three parts. Model manages the data, view handles the display & the controller acts as an intermediary between user, model & view. These separations makes applications easier to maintain & scale.

Q.3

What are testing applications & how to test an app?

→

Testing apps are specialized software tools designed to evaluate, verify & monitor other software app. They ensure app performs as intended, is free of bugs, handles heavy traffic & meets security standards.

Code and Output

2. Display "Hello World" Open:

Controllers/HomeController.:-

```
using Microsoft.AspNetCore.Mvc;

namespace HelloWebApp.Controllers
{
    public class HomeController: Controller
    {
        public IActionResult Index()
        {
            return Content("Hello World from ASP.NET Core!");
        }
    }
}
```

Output:-



2.

HomeController.cs

```
using Microsoft.AspNetCore.Mvc;

namespace HelloWebApp.Controllers
{
    public class HomeController: Controller
    {
        public IActionResult Index()
        {
            return View();
        }
        [HttpPost]
        public IActionResult Index (string name)
        {
            ViewBag.Message = "Welcome" + name; return View();
        }
    }
}
```

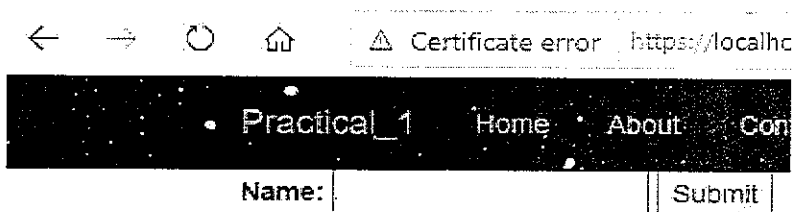
Code and Output

Index.cshtml

```
<form method="post">
    <label>Name:</label>
    <input type="text" name="name" />
    <button type="submit">Submit</button>
</form>
```

```
@if(ViewBag.Message != null)
{
    <h2>@ViewBag.Message</h2>
}
```

Output:-



WelcomeAarush

© 2026 - Practical_1

```
3.
Using System;
Using Xunit;
namespace HelloWebApp. Tests
{
    public class SimpleTests
    {
        [Fact]
        public void AdditionTest()
        {
            int result = 5+ 5;
            Assert.Equal(10, result);
        }
    }
}
```

Output:-

AdditionTest : Passed

Sr.No	Date	Topic	Sign
2	3/7/26	Creating a testing project in ASP.NET Core	Sandhya

Testing is an important part of software development. In ASP.NET Core apps, testing helps devs verify that the app works correctly & identify bugs before software is released. Visual Studio 2019 provides built-in support for creating & running test projects.

Most commonly used testing frameworks in ASP.NET Core are :

1. MSTest : Microsoft's testing framework.
2. xUnit : Most popular testing framework for ASP.NET Core applications.
3. NUnit : Another widely used unit testing framework.

Practical No.2 :- Creating Testing Projects

1.Create a Class to Test

Create a class in the main project.

Calculator.cs using System;

using System.Collections.Generic;

using System.Linq;

using System.Threading.Tasks;

namespace StudentManagement

{

public class Calculator

{

public int Add(int a, int b)

{

return a + b;

}

public int Multiply(int a, int b)

{

return a * b;

}

}

}

Practical No.2 :-
Creating Testing Projects

2.Create Unit Test Class

Inside:-StudentManagement.Tests

Create :-CalculatorTests.cs

```
using System;
```

```
using System.Collections.Generic;
```

```
using System.Text; using Xunit;
```

```
using StudentManagement;
```

```
using StudentManagement.Models;
```

```
namespace StudentManagementTests {
```

```
    public class CalculatorTests
```

```
    {
```

```
        [Fact]
```

```
        public void Add_TwoNumbers_ReturnsSum()
```

```
        {
```

```
            Calculator c = new StudentManagement.Calculator();
```

```
            int result = c.Add(10, 20);
```

```
            Assert.Equal(30, result);
```

```
        }
```

```
    }
```

```
}
```

Practical No.2 :- Creating Testing Projects

Output:-

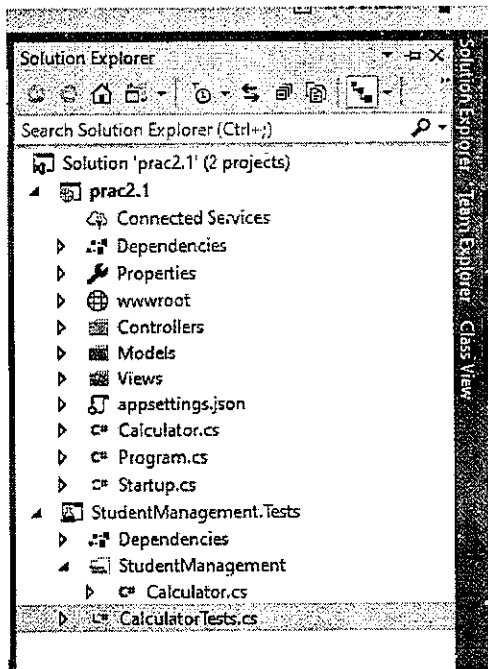
Run All Run... Playlist: All Tests		
prac2.1 (1 tests)		
StudentManagement.Tests (1)		6 ms
StudentManagement.Tests (1)		6 ms
CalculatorTests (1)		6 ms

Group Summary

Grouped by Hierarchy: StudentManagement.Tests

Duration: 0:00:00.006

1 Test Passed



prac2.1 (1 tests; 1 failed)		
StudentManagement.Tests (1)		9 ms
StudentManagement.Tests (1)		9 ms
CalculatorTests (1)		9 ms

Summary

Last Test Run Failed (Total Run Time 0:00:01.0027119)

1 Test Failed

Sr. No.	Date	Topic	Sign
3	10.20/7/26	Creating Applications with Data (CRUD)	Sandhya

CRUD (Create, Read, Update, Delete) forms the foundational architecture for managing persistent data records in enterprise web apps. In ASP.NET Core, these 4 basic data-handling operations map directly to specialized standard HTTP architectural protocols, domain models & Entity Framework (EF) Core state-tracking mechanism.

Entity Framework (EF) Core is Microsoft's lightweight, open-source & cross-platform Object-Relational Mapper (ORM). It enables developers to interact with relational & NoSQL databases using C# objects & LINQ, eliminating need to write & maintain manual data-access code or raw SQL queries.

Practical 3

Creating Applications with Data (CRUD)

Step 1 -Create Model Class (Student.cs)

```
using System.ComponentModel.DataAnnotations;

namespace StudentCRUD.Models{

public class Student {

    [Key]

    public int StudentId

    { get; set; }[Required][StringLength(50)]

    public string Name

    { get; set; }[Range(18, 60)]

    public int Age

    { get; set; }[StringLength(50)]

    public string Course

    { get; set; }

}}

```

Steps 2- Create Database Context Class(StudentContext.cs)

```
using System.Data.Entity;

namespace StudentCRUD.Models{

public class StudentContext:DbContext

{

    public StudentContext() : base("StudentConnection") {}

    public DbSet<Student> Students

    { get; set; }

}}

```

Practical 3

Creating Applications with Data (CRUD)

Step 3: Configure Connection String (Web.config)

```
<connectionStrings>
```

```
<add
```

```
  name="StudentConnection"
```

```
  connectionString="Data Source=(LocalDB)\MSSQLLocalDB; Initial  
Catalog=StudentDB; Integrated Security=True"
```

```
  providerName="System.Data.SqlClient"
```

```
/>
```

```
</connectionStrings>
```

Add after or end of the code

Step 4: Enable Migrations & Create Migration (Open Package Manager Console.)

- Enable-Migrations
- Add-Migration InitialCreate

Step 5: Create Database

Update-Database

Steps 6: Create Controller using Scaffolding (on Controller)

MVC 5 Controller with views, using Entity Framework

- Model Class : Student Data
- Context Class : StudentContext
- Controller Name : StudentsController
- Click: Add
-

Step 7 : Run the application:

Ctrl + F5

Practical 3

Creating Applications with Data (CRUD)

Output:-

Delete	Details																										
Are you sure you want to delete this? Student <table><tr><td>Name</td><td>Grishma</td></tr><tr><td>Age</td><td>19</td></tr><tr><td>Course</td><td>IT</td></tr></table> Delete Back to List © 2026 - My ASP.NET Application	Name	Grishma	Age	19	Course	IT	Student <table><tr><td>Name</td><td>Grishma</td></tr><tr><td>Age</td><td>19</td></tr><tr><td>Course</td><td>IT</td></tr></table> Edit Back to List © 2026 - My ASP.NET Application	Name	Grishma	Age	19	Course	IT														
Name	Grishma																										
Age	19																										
Course	IT																										
Name	Grishma																										
Age	19																										
Course	IT																										
Edit	Index																										
Student <table><tr><td>Name</td><td>Grishma</td></tr><tr><td>Age</td><td>19</td></tr><tr><td>Course</td><td>IT</td></tr></table> Save Back to List © 2026 - My ASP.NET Application	Name	Grishma	Age	19	Course	IT	Create New <table><thead><tr><th>Name</th><th>Age</th><th>Course</th><th></th></tr></thead><tbody><tr><td>Grishma</td><td>19</td><td>IT</td><td>Edit Details Delete</td></tr><tr><td>Shreya</td><td>20</td><td>IT</td><td>Edit Details Delete</td></tr><tr><td>Shravani</td><td>19</td><td>IT</td><td>Edit Details Delete</td></tr><tr><td>Priya</td><td>20</td><td>IT</td><td>Edit Details Delete</td></tr></tbody></table> © 2026 - My ASP.NET Application	Name	Age	Course		Grishma	19	IT	Edit Details Delete	Shreya	20	IT	Edit Details Delete	Shravani	19	IT	Edit Details Delete	Priya	20	IT	Edit Details Delete
Name	Grishma																										
Age	19																										
Course	IT																										
Name	Age	Course																									
Grishma	19	IT	Edit Details Delete																								
Shreya	20	IT	Edit Details Delete																								
Shravani	19	IT	Edit Details Delete																								
Priya	20	IT	Edit Details Delete																								

Sr.No	Date	Topic	Sign
4	20/7/26	Creating a Shopping Cart Application	Sandhya

Aim: Creating a Shopping Cart App.

To build a high-performance Shopping Cart App in ASP.NET Core, you must implement a architecture that clearly separates data models, business logic & session state persistence.

This comprehensive concept guide outlines the core architecture, data schemas, storage strategies & programmatic work flow using standard ASP.NET Core MVC & Entity Framework Core (EF Core).

Practical 4

Step 1: Create Product.cs

```
namespace ShoppingCartApp.Models
{
    public class Product
    {
        public int ProductID { get; set; } public
        string ProductName { get; set; }
        public decimal Price { get; set; } }
    }
```

Step 2: Create Cart Model (CartItem.cs)

```
namespace ShoppingCartApp.Models
{
    public class CartItem
    {
        public Product Product { get; set; }
        public int Quantity { get; set; } }
    }
```

Step 3: Create Home Controller

```
using System.Collections.Generic;
using System.Web.Mvc;
using ShoppingCartApp.Models;

namespace ShoppingCartApp.Controllers
{
    public class HomeController : Controller
    {
        public ActionResult Index()
        {
            List<Product> products = new List<Product>()
            {
                new Product{ ProductID=1, ProductName="Laptop", Price=55000 }, new
                Product{ ProductID=2, ProductName="Mouse", Price=600 }, new Product{
                ProductID=3, ProductName="Keyboard", Price=1500 }, new Product{
                ProductID=4, ProductName="Monitor", Price=12000 } };

            return View(products);
        }
    }
```

Step 4: Create Index View (Index.cshtml)

```
@model IEnumerable<ShoppingCartApp.Models.Product>
```

```
<h2>Products</h2>
```

```
<table class="table table-bordered">
```

```
<tr>
```

```
<th>ID</th>
```

```
<th>Name</th>
```

```
<th>Price</th>
```

```
<th></th>
```

```
</tr>
```

```
@foreach(var item in Model)
```

```
{
```

```
<tr>
```

```
<td>@item.ProductID</td>
```

```
<td>@item.ProductName</td>
```

```
<td>@item.Price</td>
```

```
<td>
```

```
@Html.ActionLink("Add To Cart","AddToCart",  
new{id=item.ProductID})
```

```
</td>
```

```
</tr>
```

```
}
```

```
</table>
```

Step 5: Create Cart Controller

```
using ShoppingCartApp.Models;  
using System.Collections.Generic;  
using System.Linq;  
using System.Web.Mvc;
```

```
public ActionResult AddToCart(int id)
```

```
{
```

```
List<Product> products = new List<Product>()
```

```
{
```

```
new Product{ ProductID=1, ProductName="Laptop", Price=55000 }, new
```

```
Product{ ProductID=2, ProductName="Mouse", Price=600 }, new
```

```
Product{ ProductID=3, ProductName="Keyboard", Price=1500 }, new
```

```
Product{ ProductID=4, ProductName="Monitor", Price=12000 } };
```

```
Product product = products.FirstOrDefault(x => x.ProductID == id);
```

```
List<CartItem> cart;
```

```
if (Session["Cart"] == null)
```

```
{
```

```
cart = new List<CartItem>();
```

```
}
```

```
else
```

```
{
```

```
cart = (List<CartItem>)Session["Cart"];
```

```
}
```

```
CartItem item = cart.FirstOrDefault(x => x.Product.ProductID == id);
```

```
if (item == null)
```

```

{
    cart.Add(new CartItem()
    {
        Product = product,
        Quantity = 1
    });
}
else
{
    item.Quantity++;
}

Session["Cart"] = cart;

return RedirectToAction("Index", "Home");
}

```

```

public ActionResult Index()
{
    List<CartItem> cart;

    if (Session["Cart"] == null)
    {
        cart = new List<CartItem>();
    }
    else
    {
        cart = (List<CartItem>)Session["Cart"];
    }

    return View(cart);
}

```

Step 6: Create Cart View

```

model IEnumerable<ShoppingCartApp.Models.Product>
<h2>Products</h2>
<table class="table table-bordered">
  <tr> <th>ID</th> <th>Name</th> <th>Price</th> <th></th> </tr> @foreach
(var item in Model)
  {
    <tr>
      <td>@item.ProductID</td>
      <td>@item.ProductName</td>
      <td>@item.Price</td>
      <td>@Html.ActionLink("Add To Cart", "AddToCart", "Cart", new { id =
item.ProductID }, null) </td>
      <td>@Html.ActionLink("Remove", "Remove", "Cart", new { id = item.ProductID }, null)
      </td>
    </tr>
  }
</table>

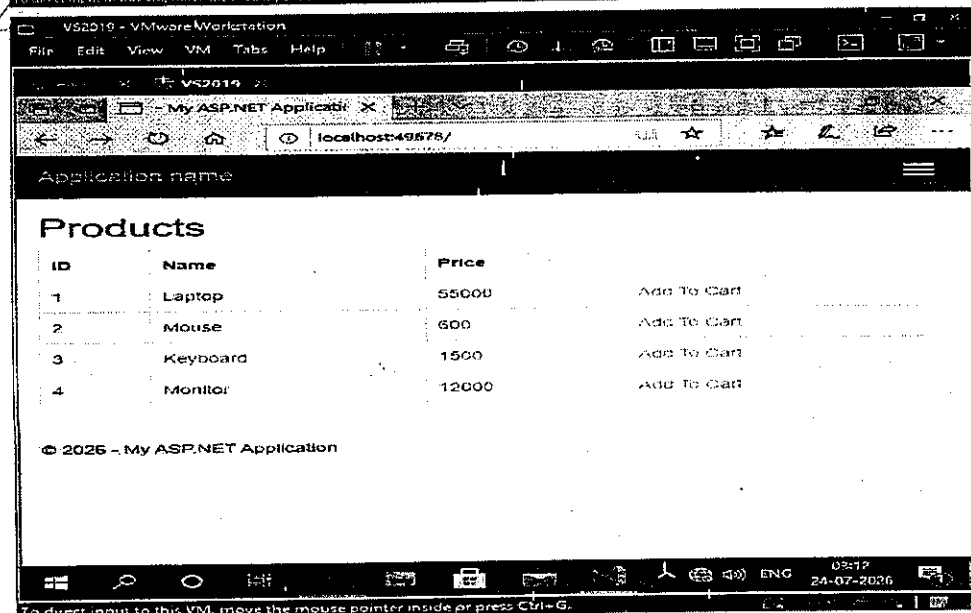
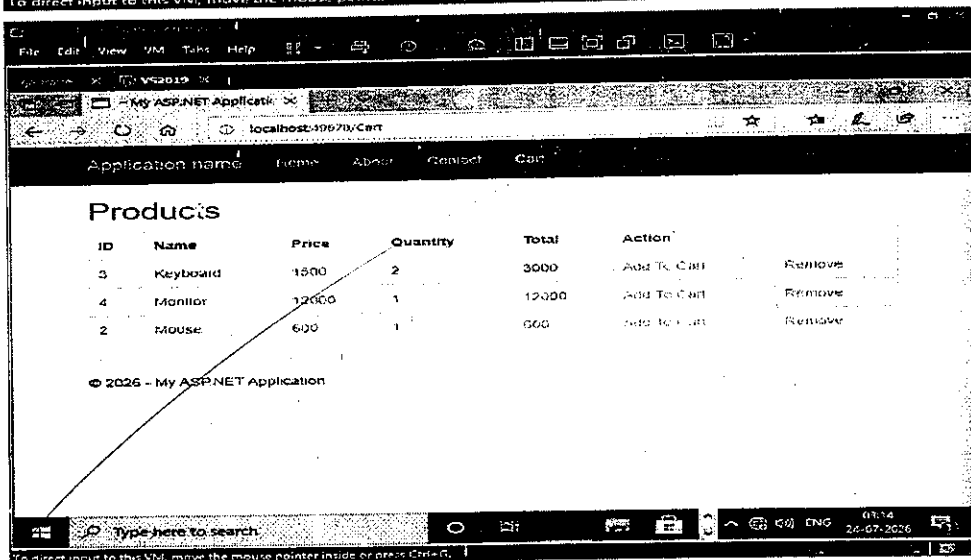
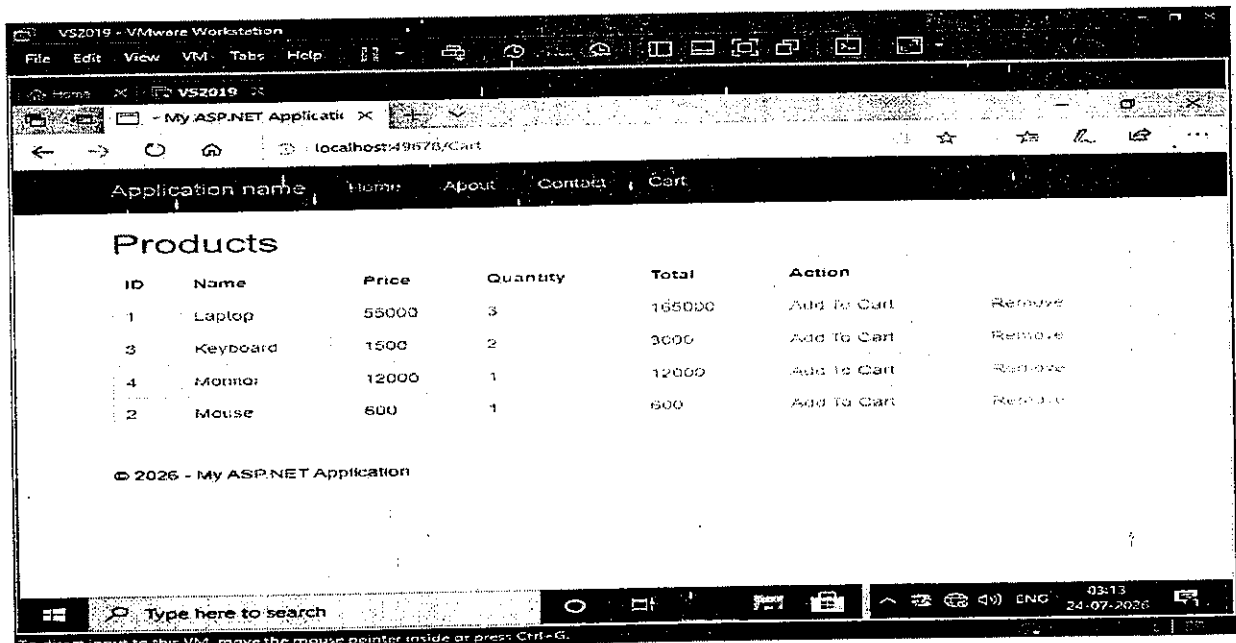
```

Step 7: Add View Cart Link

```

<li>@Html.ActionLink("Cart","Index","Cart")</li>

```



Sr. No	Date	Topic	Sign
5	7/8/26	Implementing Security to Application	Sundaram

Authentication is verifying who a user is, while authorization is determining what they are allowed to do. In ASP.NET Core, these two processes work together but are handled by distinct middleware components in app. pipeline.

- Authentication (Who are you?): The process of challenge & verification. The user provides credentials (like a username/password or a token) & sys. validates them to create an identity.
- Authorization (What can you do?): The process of checking permissions. The system looks at the authenticated user's identity, roles or claims to decide if they can access a specific resource.

Practical 5

Step 1: Create a Secure Controller(StudentController.cs)

```
namespace SecureMVCAApp.Controllers
{
    [Authorize]
    public class StudentController : Controller
    {
        public ActionResult Index()
        {
            return View();
        }
    }
}
```

Step 2: Create View(Index.cshtml)

```
<h2>Student Dashboard</h2>
<h3>Only logged-in users can see this page.</h3>
```

Step 3: Protect Only One Action(StudentController.cs)

```
public class StudentController : Controller
{
    public ActionResult About()
    {
        return View();
    }
    [Authorize]
    public ActionResult Result()
    {
        return View();
    }
}
```

Now

<https://localhost:44320/Student/Index>

<https://localhost:44320/Student/about>

Everyone can access.

<https://localhost:44320/Student/Result>

Redirect to Login page

Step 4: Create View(Index.cshtml)

[Authorize]

```
public class StudentController : Controller
{
```

```
    [AllowAnonymous]
```

```
    public ActionResult About()
```

```
    {
```

```
        return View();
```

```
    }
```

```
    public ActionResult Dashboard()
```

```
    {
```

```
        return View();
```

```
    }
```

```
}
```

Log in.

Use a local account to log in

Use another service to log in

Email

student@studnet.com

Password

☐ Remember this

Log in

Forgot a password?

© 2026 - My ASP.NET application

There are no external authentication providers configured.
See this article for details on setting up the ASP.NET application to support logging in via external providers.

About.

Your application description page.

Use this area to provide additional information.

© 2026 - My ASP.NET Application

Studnet Result

Only Logged-in users can see this page.

© 2026 - My ASP.NET Application

Studnet Dashboard

Only Logged-in users can see this page.

© 2026 - My ASP.NET Application